

目录

×

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

Search

原创

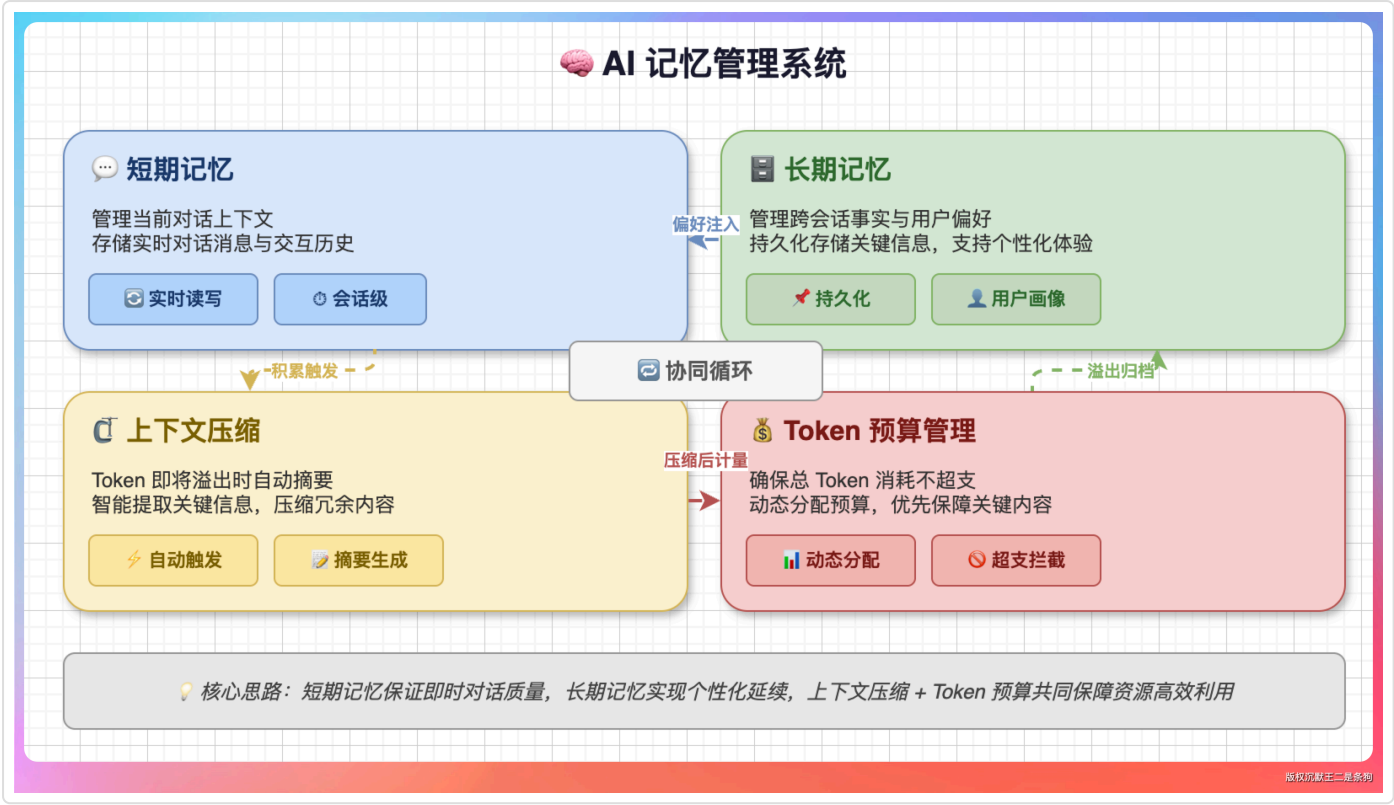
短期记忆 + 长期记忆 + Context 压缩，让有了 Memory 的 Agent 不再失忆。

大家好，我是二哥呀。

PaiCLI 第二期做完 Plan-Execute 之后，有个问题一直让我很头疼——Agent 的记忆力跟金鱼一样，聊着聊着就忘了前面说过什么。

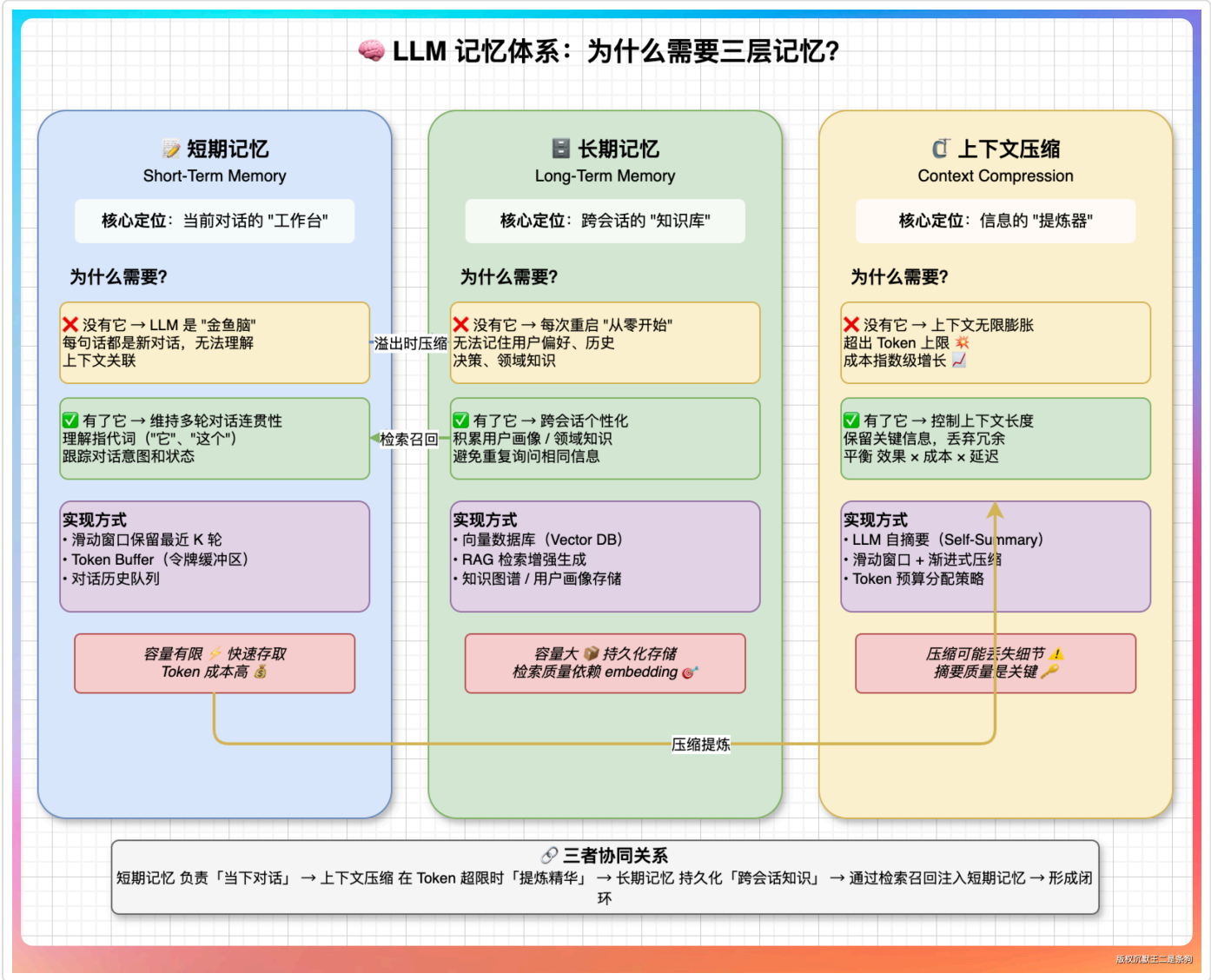
你跟它说“我喜欢用 JDK 17”，清空对话再来一轮，它又傻x地给你生成 JDK 8 的代码。

这期就来解决这个问题，给 Agent 加上完整的 Memory 系统。



整个 Memory 分三块。

短期记忆，管的是当前对话的上下文——用户说了啥、工具返回了啥、Agent 做了哪些决策。LLM 本身是无状态的，每次请求都是独立的，根本不记得上一轮聊了什么。短期记忆就是替它“记着”，下次输入的时候把历史消息一起带上。



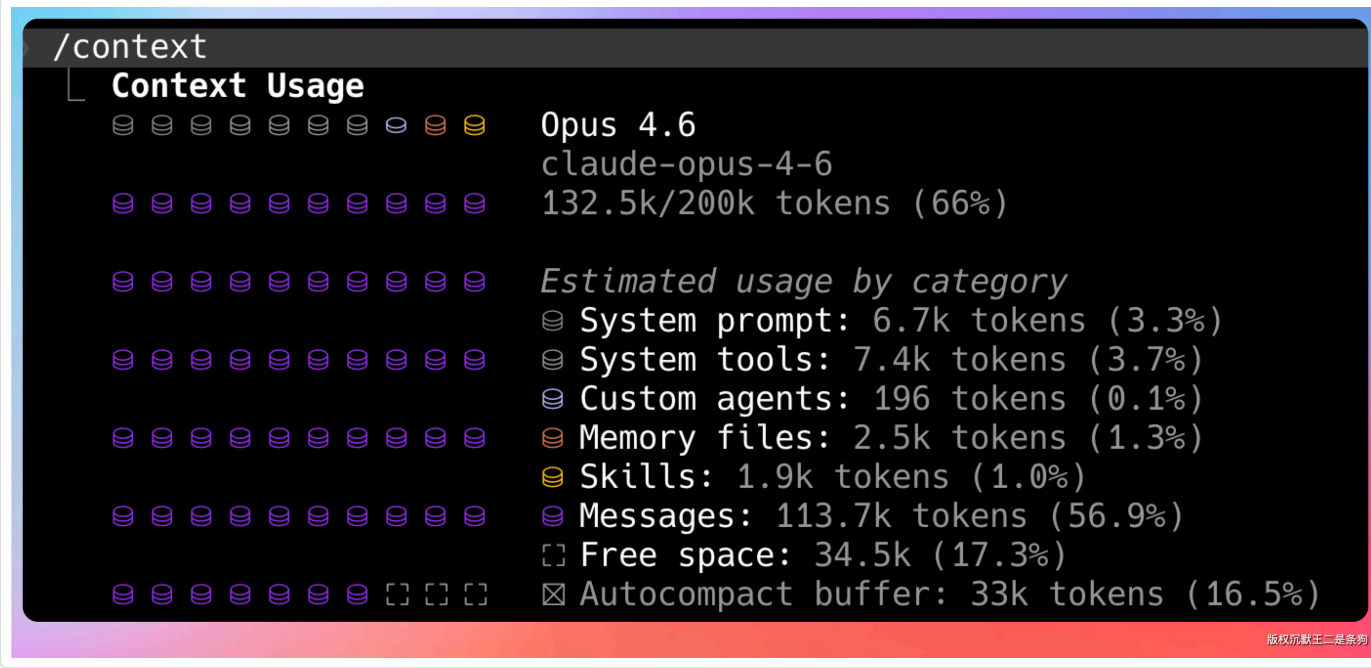
但短期记忆有个致命缺点——会话一关，全没了。下次启动 Agent，它完全不知道你之前的偏好、项目用什么技术栈、代码风格有什么约定。

所以得有**长期记忆**，把这些跨会话的关键信息持久化到磁盘上，不管开多少个会话窗口，Agent 都能记住。

那问题又来了，上下文窗口是有限的。Claude Opus 4.6 默认 200K，听着很大，但短期记忆加长期记忆一股脑全塞进去，不光浪费 token，不相关的信息还会干扰 LLM 的判断。

目录

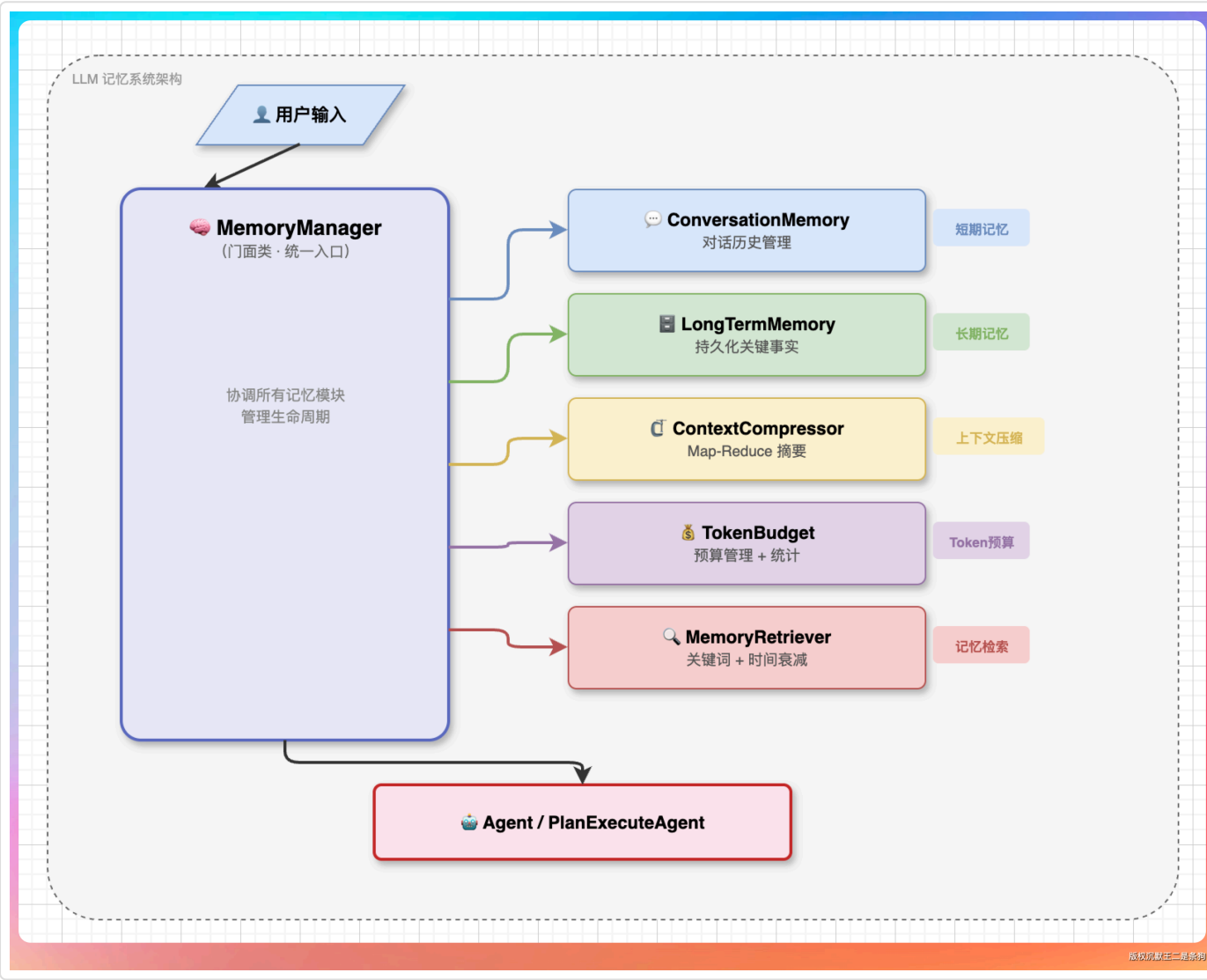
- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)**
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)



所以需要**上下文压缩**——达到阈值就做摘要，保留关键信息，丢掉冗余细节。这样 LLM 才有足够的空间思考，手里拿到的也都是有用的上下文。

01、Memory 的整体设计

先看全貌。



MemoryManager 是整个系统的门面，底下管着 ConversationMemory、LongTermMemory、ContextCompressor、TokenBudget、MemoryRetriever 五个组件。Agent 不用关心这些细节，对外就暴露两个操作——存消息、取记忆。存的时候自动管预算，取的时候自动检索相关上下文。

02、记忆的基本单元

记忆不是一堆字符串往列表里塞就完事了。Agent 得知道每条记忆是什么类型——对话还是事实？

什么时候产生的——用来做时间衰减。

占多少 token——用来做预算管理。有没有附加信息——比如这条事实来自哪个项目。

所以每条记忆条目长这样：

目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)**
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

java 复制代码

```
public class MemoryEntry {
    private final String id;
    private final String content;
    private final MemoryType type;
    private final Instant timestamp;
    private final Map<String, String> metadata;
    private final int tokenCount;

    public enum MemoryType {
        CONVERSATION, // 对话记忆
        FACT,          // 事实记忆（用户偏好、项目信息）
        SUMMARY,       // 摘要记忆
        TOOL_RESULT    // 工具执行结果
    }
}
```

CONVERSATION 是对话消息，FACT 是用户偏好、项目信息这类关键事实，SUMMARY 是压缩后的摘要，TOOL_RESULT 是工具执行结果。

TOOL_RESULT 之所以单独分出来，是因为工具返回的内容通常特别长（比如读一个文件能返回几百行），但检索的时候又需要知道“之前执行过什么命令”。标记出来之后，压缩时可以对工具结果更激进地砍，对话内容则多保留一些语义。

token 估算也得有个方法，预算管理全靠它：

java 复制代码

```
public static int estimateTokens(String text) {
    if (text == null || text.isEmpty()) return 0;
    long chineseChars = text.chars()
        .filter(c -> c > 0x4E00 && c < 0x9FFF).count();
    long otherChars = text.length() - chineseChars;
    return (int) Math.ceil(chineseChars / 1.5 + otherChars / 4.0);
}
```

中文约 1.5 字一个 token，英文约 4 个字符一个 token。精确计算得用 tokenizer。

04、短期记忆

短期记忆干两件事：**存消息**和**自动淘汰**。

token 预算是有限的，聊多了总会撑满。这时候最简单的策略就是淘汰最旧的消息——跟操作系统的页面置换一个道理，内存不够了就把最久没用的页面换出去。对话是顺序的，最旧的消息通常最不重要，FIFO 就够了。

java 复制代码

```
public class ConversationMemory implements Memory {
    private final LinkedHashMap<String, MemoryEntry> entries;
    private final int maxTokens;
    private final AtomicInteger currentTokens;
    private final List<MemoryEntry> compressedSummaries;

    @Override
    public void store(MemoryEntry entry) {
        entries.put(entry.getId(), entry);
        currentTokens.addAndGet(entry.getTokenCount());

        // 超出预算时自动淘汰最旧的条目
        while (currentTokens.get() > maxTokens && entries.size() > 1) {
            evictOldest();
        }
    }
}
```

被淘汰的消息不会直接扔掉，而是放进 **compressedSummaries** 列表，等着后面压缩成摘要——信息还在，只是换了个更紧凑的形态。

getUsageRatio() 返回当前 token 使用率，超过 80% 的时候 MemoryManager 就会自动触发 ContextCompressor 来做压缩。

目录

[01、Memory 的整体设计](#)

[02、记忆的基本单元](#)

[04、短期记忆](#)

[05、长期记忆](#)

[06、上下文压缩](#)

[07、Token 预算管理](#)

[08、记忆检索](#)

[09、集成到 Agent](#)

[10、MemoryManager](#)

[11、跑起来看看效果](#)

[PaiCLI 如何写到简历上？](#)

05、长期记忆

短期记忆跟着会话走，对话一关就没了。但有些信息是跨会话都需要的——“用户喜欢 JDK 17”“项目用 Maven 构建”——这些东西得存到磁盘上，下次启动 Agent 还能用。

java 复制代码

```
public class LongTermMemory implements Memory {
    private static final String STORAGE_DIR = ".paicli/memory";
    private static final String STORAGE_FILE = "long_term_memory.json";

    public LongTermMemory() {
        // 启动时从磁盘加载
        loadFromDisk();
    }

    @Override
    public void store(MemoryEntry entry) {
        // 去重检查：内容完全相同则跳过
        Optional<Map.Entry<String, MemoryEntry>> existing = entries.entrySet().stream()
            .filter(e -> e.getValue().getContent().equals(entry.getContent()))
            .findFirst();
        if (existing.isPresent()) return;

        entries.put(entry.getId(), entry);
        tokenCounter.addAndGet(entry.getTokenCount());
        saveToDisk(); // 每次存完都持久化
    }
}
```

这里有三个设计点值得说一下。

首先是**自动去重**，用户连续三次说“我喜欢用 Java”，长期记忆里不会存三遍，内容相同的直接跳过。

然后是**即时持久化**，每次 `store` 都调 `saveToDisk`，写入 `~/.paicli/memory/long_term_memory.json`。存储路径支持三种配置——默认路径、JVM 参数 `-Dpaicli.memory.dir=/path`、环境变量 `PAICLI_MEMORY_DIR`，方便测试环境和生产环境隔离。

最后是**启动时加载**，Agent 启动的时候自动把之前的记忆读进来。加载时会保留原始时间戳，这一点很重要——如果时间戳被覆盖成当前时间，后面做时间衰减检索的时候，旧记忆就不会被正确衰减了。

检索这块，用了 jieba 分词器做中文优先的轻量匹配。“项目技术栈”能正确切成“项目”“技术”两个词，命中率比单纯的字符串 `contains` 高不少。

java 复制代码

```
final class MemoryQueryTokenizer {
    private static final JiebaSegmenter SEGMENTER = new JiebaSegmenter();

    static Set<String> tokenize(String query) {
        LinkedHashSet<String> tokens = new LinkedHashSet<>();
        List<String> words = SEGMENTER.sentenceProcess(
            query.toLowerCase(Locale.ROOT).trim());
        for (String word : words) {
            String trimmed = word.trim();
            // 过滤单字符和纯标点
            if (trimmed.length() >= 2 && !isPunctuation(trimmed)) {
                tokens.add(trimmed);
            }
        }
        return tokens;
    }
}
```

jieba 在 Java 生态里算比较成熟的中文分词库了。分完词后还会过滤掉单字和纯标点——中文里“的”“了”“是”这种单字基本没有检索价值，留着只会带来噪音。

`search` 方法调用分词器，同时匹配内容和 `metadata`：

目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)**
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

```
public List<MemoryEntry> search(String query, int limit) {
    Set<String> queryTokens = MemoryQueryTokenizer.tokenize(query);
    return entries.values().stream()
        .filter(entry -> {
            if (MemoryQueryTokenizer.matches(entry.getContent(), queryTokens)) {
                return true;
            }
            return entry.getMetadata().values().stream()
                .anyMatch(value -> MemoryQueryTokenizer.matches(value,
                    queryTokens));
        })
        .limit(limit)
        .collect(Collectors.toList());
}
```

MemoryRetriever 里的 `computeRelevanceScore` 也是用 jieba 来算关键词匹配度的。

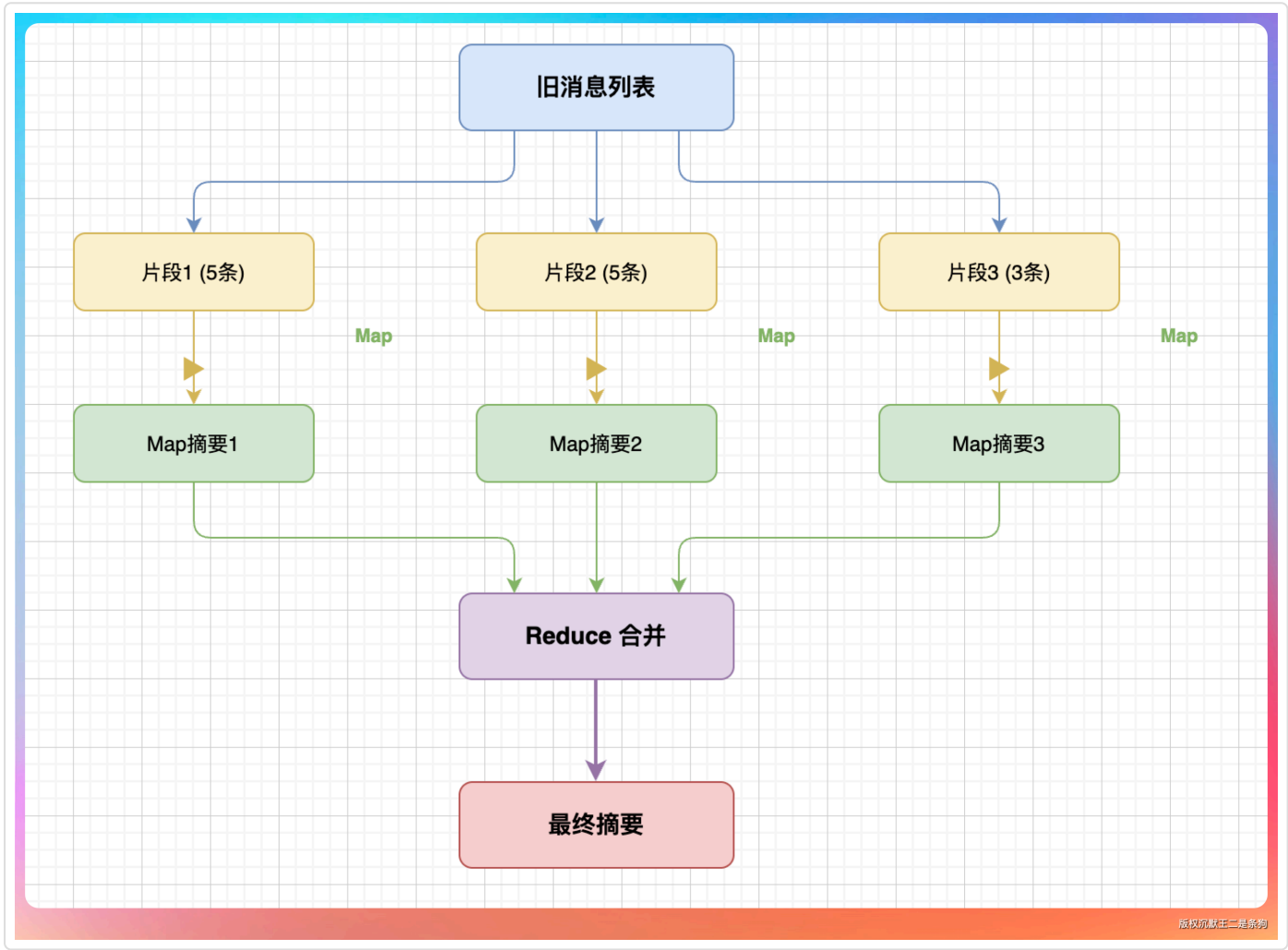
不过说实话，jieba 分词加关键词匹配跟向量检索比起来还是差了一截。“Java 框架”匹配不到“Spring Boot”，这种语义层面的理解需要 Embedding 才能搞定，我们放到第四期。

06、上下文压缩

这块是整个 Memory 系统里我觉得最有意思的部分。

短期记忆满了，旧消息得淘汰，但关键信息不能丢。怎么办？压缩成摘要，再注入回去。

压缩策略用的是 **Map-Reduce**，跟处理大文档的思路一样：



Map 阶段：把旧消息分成每 5 条一组，每组独立调用 LLM 生成摘要。分组处理比一股脑扔几十条给 LLM 效果好得多，模型处理短文本的摘要质量明显更高。

Reduce 阶段：把多个分片摘要合并成最终摘要。只有一片的话直接用，不再多调一次 LLM。

目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)**
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

java 复制代码

```
public String compress(ConversationMemory memory) {
    List<MemoryEntry> allEntries = memory.getAll();
    // 分割：旧消息 vs 近期消息（必须拷贝，因为后面会 clear 底层集合）
    int splitPoint = allEntries.size() - retainRecentRounds;
    List<MemoryEntry> oldEntries = new ArrayList<>(allEntries.subList(0, splitPoint));
    List<MemoryEntry> recentEntries = new ArrayList<>(allEntries.subList(splitPoint,
allEntries.size()));

    // Map 阶段
    List<String> chunkSummaries = mapPhase(oldEntries);

    // Reduce 阶段
    String finalSummary = chunkSummaries.size() == 1
        ? chunkSummaries.get(0)
        : reducePhase(chunkSummaries);

    // 清空旧记忆，注入摘要，保留近期记忆
    memory.clear();
    memory.store(new MemoryEntry(
        "summary-" + UUID.randomUUID().toString().substring(0, 8),
        "[历史对话摘要] " + finalSummary,
        MemoryEntry.MemoryType.SUMMARY, null,
        MemoryEntry.estimateTokens(finalSummary)
    ));
    for (MemoryEntry entry : recentEntries) memory.store(entry);

    return finalSummary;
}
```

`retainRecentRounds` 这个参数很关键——最近 3 轮消息不参与压缩，原样保留。刚聊的内容通常是当前任务的核心上下文，压缩可能会丢掉关键细节。

ContextCompressor 还干了一件事：**事实提取**。对话结束的时候，调用 `extractFacts` 让 LLM 从对话里提炼关键信息——用户偏好、项目配置、重要决策——自动塞进长期记忆。

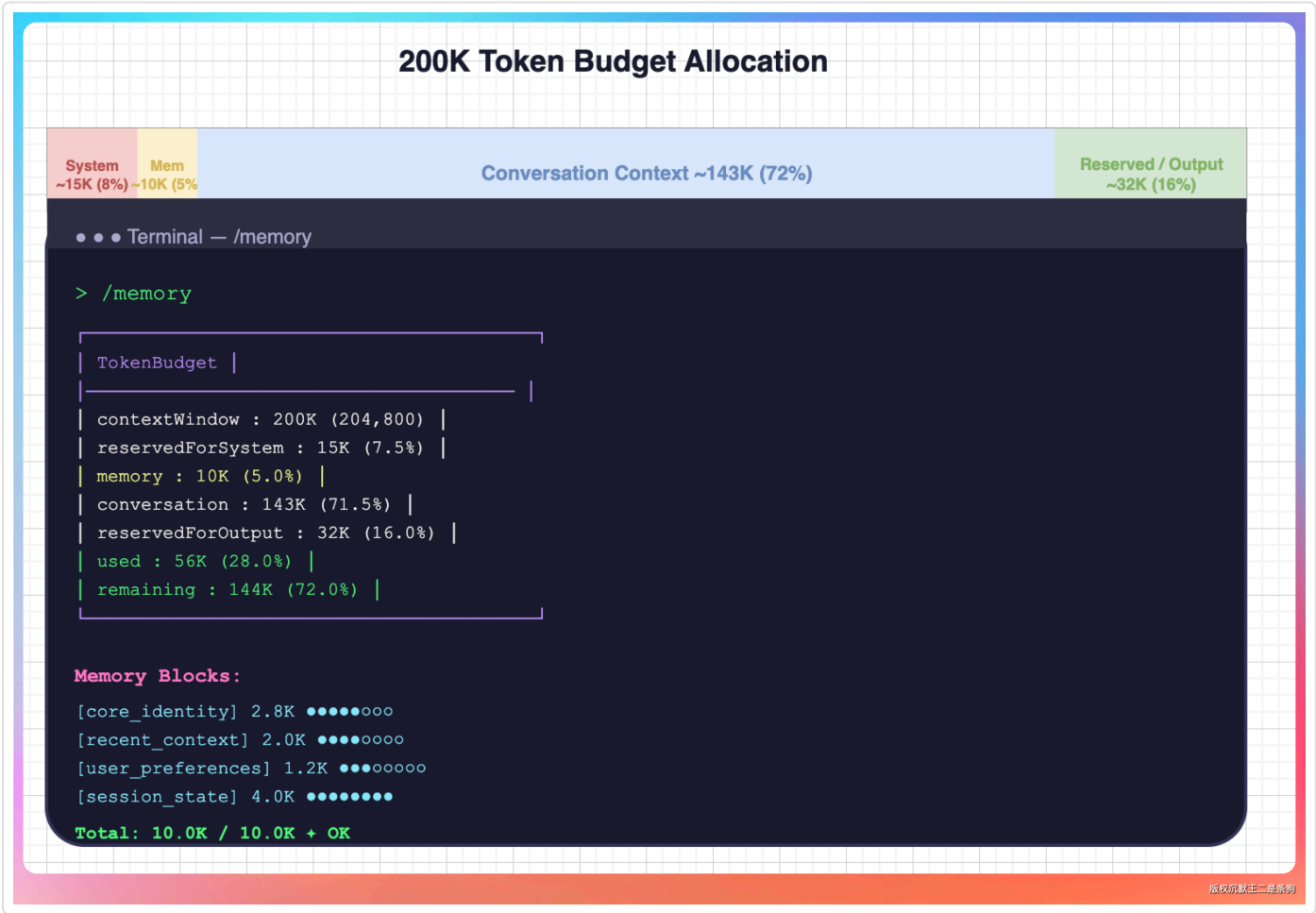
java 复制代码

```
public List<String> extractFacts(List<MemoryEntry> entries,
                                LongTermMemory longTermMemory) {

    // 用 LLM 从对话中提取关键事实
    String prompt = String.format(EXTRACT_FACTS_PROMPT, conversation);
    GLMClient.ChatResponse response = llmClient.chat(messages, List.of());

    // 解析事实，逐条存入长期记忆
    for (String fact : facts) {
        longTermMemory.store(new MemoryEntry(..., MemoryType.FACT, ...));
    }
    return facts;
}
```

比如用户说了句“我喜欢用 Spring Boot”，下次启动 Agent 的时候，这条偏好已经在长期记忆里等着了，不用再重复说一遍。



目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)**
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

07、Token 预算管理

这玩意儿不是一个简单的计数器，而是一个**预算分配器**。

模型的上下文窗口就那么大，GLM-5.1 是 200K。但这 200K 不能全给对话历史用，系统提示词要占一块，工具定义要占一块，模型生成回复也得留出空间。

java 复制代码

```
public class TokenBudget {
    private final int contextWindow;      // 200000
    private final int reservedForSystem;  // 500
    private final int reservedForTools;   // 800
    private final int reservedForResponse; // 2000

    public int getAvailableForConversation() {
        return contextWindow - reservedForSystem
            - reservedForTools - reservedForResponse;
        // 200000 - 500 - 800 - 2000 = 196700
    }

    public boolean needsCompression(ConversationMemory memory) {
        return memory.getTokenCount()
            > getAvailableForConversation() * 0.8;
    }
}
```

对话历史超过 80% 预算就触发压缩。留 20% 余量是因为 token 估算本身就不精确（我们用的是字符数估算，不是真正的 tokenizer），卡太死容易翻车。

TokenBudget 还顺带记录了每次 LLM 调用的 token 消耗，调试的时候很好用：

java 复制代码

```
public void recordUsage(int inputTokens, int outputTokens) {
    totalInputTokens += inputTokens;
    totalOutputTokens += outputTokens;
    llmCallCount++;
}

public String getUsageReport() {
    return String.format(
        "Token 统计：调用 %d 次 | 总输入：%d | 总输出：%d",
        llmCallCount, totalInputTokens, totalOutputTokens);
}
```

在 CLI 里输入 `/memory` 就能看到完整的状态报告。

08、记忆检索

短期记忆和长期记忆都有了，但 Agent 处理新输入的时候，不能把 100 条记忆全塞给 LLM——大部分跟当前任务根本不沾边，纯属浪费 token。

MemoryRetriever 干的就是从两层记忆里把最相关的条目捞出来。

java 复制代码

```
public List<MemoryEntry> retrieve(String query, int limit) {
    List<ScoredEntry> scored = new ArrayList<>();

    // 从短期记忆中检索
    for (MemoryEntry entry : shortTermMemory.getAll()) {
        double score = computeRelevanceScore(entry, query);
        if (score > 0) scored.add(new ScoredEntry(entry, score, true));
    }

    // 从长期记忆中检索（权重 ×1.2，因为更精炼）
    for (MemoryEntry entry : longTermMemory.getAll()) {
        double score = computeRelevanceScore(entry, query) * 1.2;
        if (score > 0) scored.add(new ScoredEntry(entry, score, false));
    }

    return scored.stream()
        .sorted(Comparator.comparingDouble(ScoredEntry::score).reversed())
        .limit(limit)
        .map(ScoredEntry::entry)
        .collect(Collectors.toList());
}
```

目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)**
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

相关度计算考虑了三个维度：**关键词匹配**——把查询分词后逐词匹配，命中越多分数越高；**时间衰减**——24 小时内从 1.0 线性衰减到 0.5，三天前的旧事权重自然就低了；**来源加权**——长期记忆的条目是经过提取和精炼的，信息密度更高，给 1.2 倍权重。

检索结果通过 `buildContextForQuery` 拼成文本，注入到 LLM 的输入里。Agent 那边改动很小，构建消息时带上记忆上下文就行。



09、集成到 Agent

组件都写好了，接下来把它们插到 Agent 和 PlanExecuteAgent 里。核心改动在 `Agent.run` 方法：

```
public String run(String userInput) {  
    // 1. 存入短期记忆  
    memoryManager.addUserMessage(userInput);  
  
    // 2. 检索相关长期记忆，注入到 system prompt  
    String memoryContext = memoryManager.buildContextForQuery(userInput, 500);  
    updateSystemPromptWithMemory(memoryContext);  
  
    // 3. 添加用户输入到历史（保持原文，不污染 user message）  
    conversationHistory.add(GLMClient.Message.user(userInput));  
  
    // ... ReAct 循环 ...  
}
```

用户输入进来，先存短期记忆，再从长期记忆里捞相关上下文，注入到 system prompt，然后正常走 ReAct 循环。

这里有个细节——记忆上下文是注入到 system prompt，不是拼到 user message 里。如果塞到 user message，LLM 可能把记忆内容当成用户的指令来执行，这就乱套了。放到 system prompt 里，模型会把它当背景信息参考，不会跟用户输入混淆。

```
private void updateSystemPromptWithMemory(String memoryContext) {  
    if (memoryContext == null || memoryContext.isEmpty()) {  
        // 恢复原始 system prompt  
        conversationHistory.set(0, GLMClient.Message.system(System.PROMPT));  
    } else {  
        String enrichedPrompt = System.PROMPT + "\n" + memoryContext;  
        conversationHistory.set(0, GLMClient.Message.system(enrichedPrompt));  
    }  
}
```

目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

没有相关记忆的时候恢复原始 system prompt，有记忆就追加到末尾。干净利落。

工具执行结果也同步存入记忆：

java 复制代码

```
// 执行工具后
memoryManager.addToolResult(toolName, toolResult);
```

助手回复也存：

java 复制代码

```
// Agent 回复后
memoryManager.addAssistantMessage(response.content());
memoryManager.recordTokenUsage(response.inputTokens(),
                                response.outputTokens());
```

用户输入 `/clear` 清空对话的时候，也不是简单粗暴地全删——先把关键事实提取到长期记忆，再清空：

java 复制代码

```
public void clearHistory() {
    memoryManager.extractAndSaveFacts(); // 先保存
    // 再清空
    conversationHistory.clear();
    conversationHistory.add(systemMsg);
    memoryManager.getShortTermMemory().clear();
}
```

PlanExecuteAgent 也做了类似的集成。执行每个 Task 的时候注入长期记忆上下文，LLM 就能参考之前的项目信息做决策。执行完整个计划后，自动提取关键事实到长期记忆，不用等用户手动 `/clear`。

CLI 也新增了两个命令：`/memory` 查看记忆系统的完整状态，`/save 事实内容` 手动保存关键事实到长期记忆。

10、MemoryManager

最后说说 MemoryManager，Agent 只跟它打交道，底下那五个组件怎么协作的，Agent 根本不用管。

java 复制代码

```
public class MemoryManager {
    private final ConversationMemory shortTermMemory;
    private final LongTermMemory longTermMemory;
    private final ContextCompressor compressor;
    private final MemoryRetriever retriever;
    private final TokenBudget tokenBudget;

    // 存用户消息
    public void addUserMessage(String content) {
        shortTermMemory.store(entry);
        compressIfNeeded(); // 自动检查是否需要压缩
    }

    // 检索相关记忆
    public String buildContextForQuery(String query, int maxTokens) {
        return retriever.buildContextForQuery(query, maxTokens);
    }

    // 系统状态
    public String getSystemStatus() {
        return shortTermMemory.getStatusSummary() + "\n"
            + longTermMemory.getStatusSummary() + "\n"
            + tokenBudget.getUsageReport();
    }
}
```

`compressIfNeeded` 是自动触发压缩的关键——每次存消息后检查 token 使用率，超过 80% 就调 ContextCompressor。Agent 代码里只管存消息，压不压缩、怎么压缩，全是 MemoryManager 内部的事。

目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)**
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

门面模式的好处就在这里。如果没有 MemoryManager，Agent 得自己管短期记忆、长期记忆、压缩时机、token 预算.....代码会乱成一锅粥。

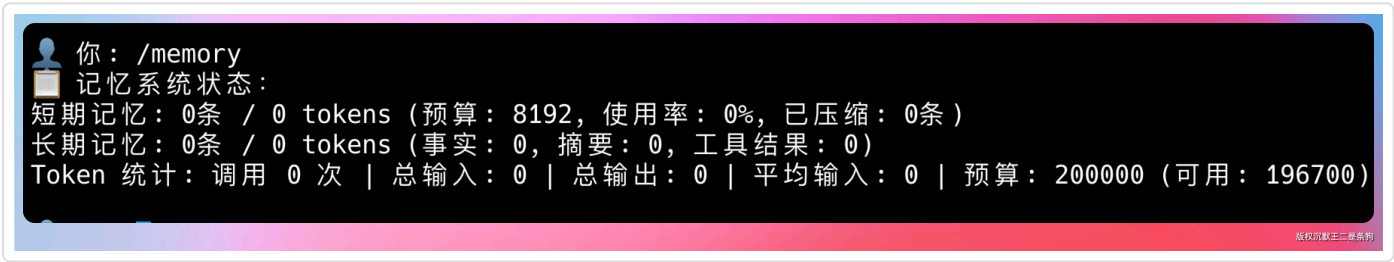
`extractAndSaveFacts` 也值得提一句。对话结束的时候（用户输入 `/clear` 或者直接退出），它会自动从当前对话里提取关键事实存入长期记忆，全程不需要用户操作。当然，用 `/save 事实内容` 手动保存也行，有些信息用户自己知道是重要的，主动存更靠谱。

11、跑起来看看效果

代码写完了，得跑起来验证一下。



第一次运行的时候记忆是空的，用着用着就会慢慢积累起来。



提示词：帮我创建一个 Java 项目叫 myapp

目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)

```
你：帮我创建一个Java项目叫myapp

思考中...

执行工具：create_project
参数：{"name":"myapp","type":"java"}
结果：项目已创建：myapp（类型：java）

执行工具：list_dir
参数：{"path":"myapp"}
结果：目录内容：
[F] pom.xml
[D] src

执行工具：list_dir
参数：{"path":"myapp/src"}
结果：目录内容：
[D] main

执行工具：list_dir
参数：{"path":"myapp/src/main"}
结果：目录内容：
[D] resources
[D] java

执行工具：list_dir
参数：{"path":"myapp/src/main/java"}
结果：目录内容：

执行工具：read_file
参数：{"path":"myapp/pom.xml"}
结果：文件内容：
<?xml version="1.0" encoding="UTF-8"?>
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
```

提示词：JDK 可以保持 17，你记一下

```
你：JDK 可以保持 17，你记一下

main dict load finished, time elapsed 326 ms
model load finished, time elapsed 16 ms.
思考中...

Token使用：输入=1152，输出=143

Agent：好的，我已经记住了！✅

**JDK 版本保持为 17**，后续在配置项目或编写代码时，我会确保使用 JDK 17 的兼容设置。比如在 `pom.xml` 中会设置：

```xml
<maven.compiler.source>17</maven.compiler.source>
<maven.compiler.target>17</maven.compiler.target>
```

需要我现在就帮你把 `myapp` 项目的 `pom.xml` 补上 JDK 17 的配置吗？
```

输入 `/clear`，提取关键事实到长期记忆。

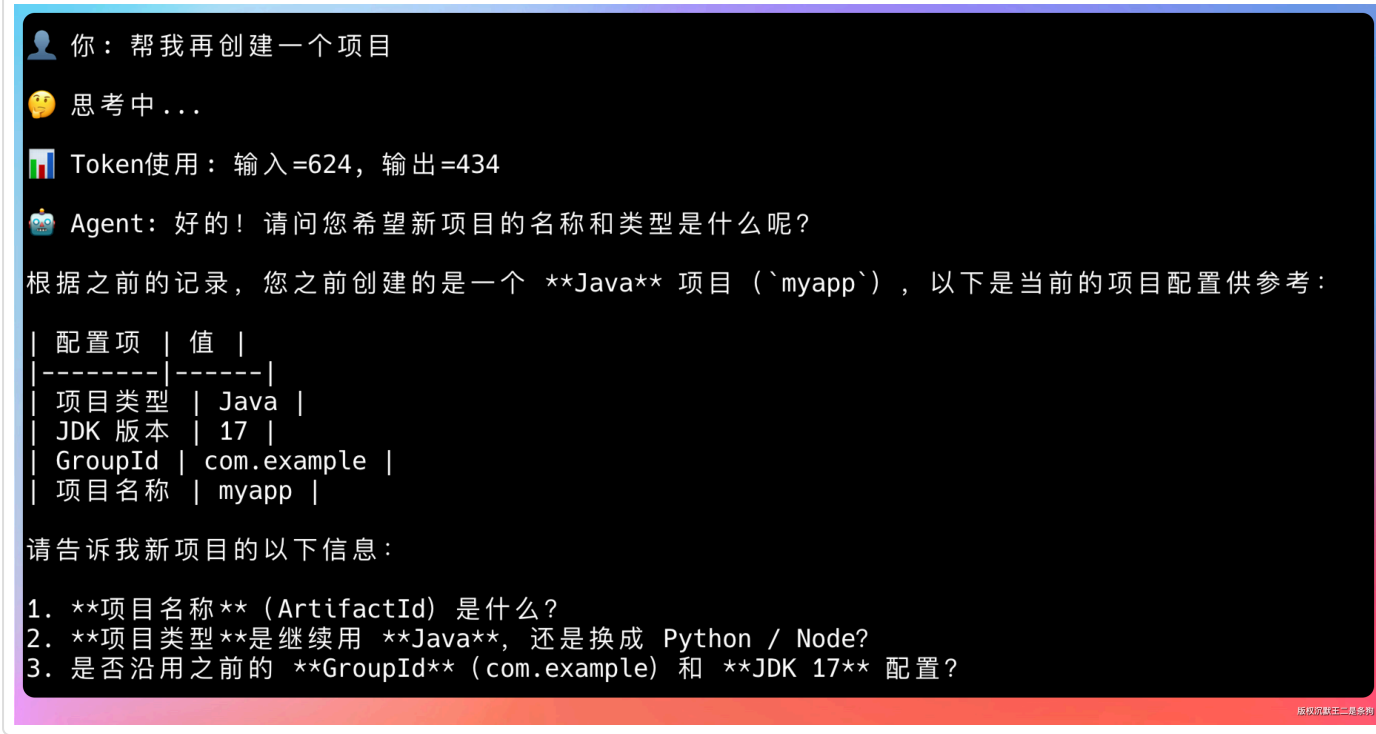
```
你：/clear
提取关键事实到长期记忆...
提取了 7 条事实
对话历史已清空，关键事实已保存到长期记忆

你：/memory
记忆系统状态：
短期记忆：0条 / 0 tokens（预算：8192，使用率：0%，已压缩：0条）
长期记忆：7条 / 51 tokens（事实：7，摘要：0，工具结果：0）
Token 统计：调用 2 次 | 总输入：1957 | 总输出：313 | 平均输入：979 | 预算：200000（可用：196700）
```

提示词：帮我再创建一个项目

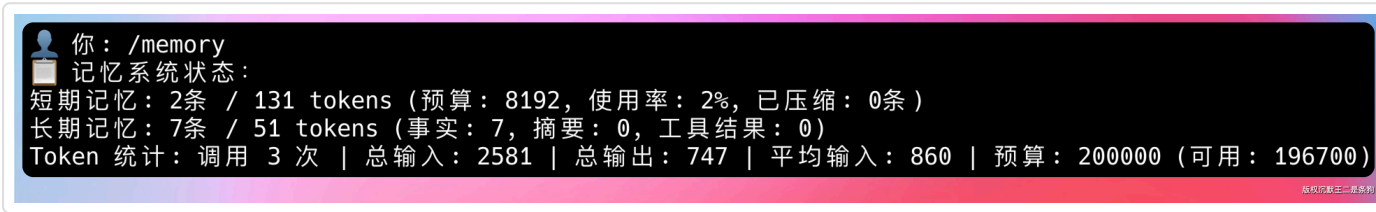
目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)**
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)



看到没？清空对话之后，Agent 还记得你喜欢 JDK 17。检索器自动把这条事实从长期记忆里捞了出来，注入到上下文中，Agent 的决策直接就变准了。

再看看 `/memory` 命令的输出：



短期记忆用了多少 token、长期记忆存了几条事实、LLM 调了几次、token 消耗多少——全在这了。

手动存事实也行，`/save` 项目使用Maven构建：



下次对话的时候，Agent 就知道你用的是 Maven，不会傻傻地生成 Gradle 的配置了。

说到底，Agent 的记忆系统就管三件事：**记住什么、忘掉什么、想起什么**。短期记忆负责“记住”，上下文压缩负责“该忘就忘”，长期记忆加检索器负责“关键时刻想得起来”。

下一期接入 RAG 检索，让 Agent 能读懂整个代码库。

项目代码已更新至：github.com/itwanger/paicli

PaiCLI 如何写到简历上？

PaiCLI 项目（第 3 期） | 2026.04 - 2026.05 | 独立开发者

项目描述：为 Agent CLI 加入 Memory 系统，实现短期记忆自动管理、长期记忆持久化、Map-Reduce 上下文压缩和 Token 预算控制。

核心职责：

- 设计 Memory 系统，实现 4 种记忆类型（对话/事实/摘要/工具结果）；实现短期记忆，超出 token 阈值时自动淘汰最旧条目，保留压缩摘要等待重新注入上下文
- 开发长期记忆，支持 JSON 文件持久化和 Agent 启动时自动加载，实现内容去重和关键词检索，跨会话保留用户偏好和项目信息
- 实现上下文压缩器，采用 Map-Reduce 策略分片摘要后合并，保留最近 3 轮完整消息不参与压缩，并支持自动提取关键事实到长期记忆
- 实现预算管理和记忆检索，基于 jieba 分词+关键词匹配+时间衰减+来源加权来排序检索结果

目录

[01、Memory 的整体设计](#)

[02、记忆的基本单元](#)

[04、短期记忆](#)

[05、长期记忆](#)

[06、上下文压缩](#)

[07、Token 预算管理](#)

[08、记忆检索](#)

[09、集成到 Agent](#)

[10、MemoryManager](#)

[11、跑起来看看效果](#)

[PaiCLI 如何写到简历上？](#)



2人已点赞



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

目录

- [01、Memory 的整体设计](#)
- [02、记忆的基本单元](#)
- [04、短期记忆](#)
- [05、长期记忆](#)
- [06、上下文压缩](#)**
- [07、Token 预算管理](#)
- [08、记忆检索](#)
- [09、集成到 Agent](#)
- [10、MemoryManager](#)
- [11、跑起来看看效果](#)
- [PaiCLI 如何写到简历上？](#)